Ho Ting (Bosco) Hung[1]

January 8, 2026

Westerlund: Westerlund ECM Panel Cointegration Test

Abstract: This paper introduces a new Python and R package: Westerlund. The Python and R codes implement the Westerlund (2007) Panel Cointegration Tests, which are used to determine if there is a long-run relationship (cointegration) between a dependent variable y and a set of independent variables x across a panel of N cross-sectional units over T time periods. The code provides a functional approximation of the Stata command `xtwest` developed by Persyn and Westerlund (2008) and also offers an intuitive visualization diagnostic tool to facilitate the examination of the bootstrap distribution and significance of the statistics of interest.

[1]Department of Politics and International Relations, University of Oxford
bosco.hung@st-annes.ox.ac.uk

Contents

1 Overview

Econometric models, such as the Autoregressive Distributed Lag (ARDL) models, are increasingly used to analyze the long-run equilibrium relationship between a dependent variable y and a set of independent variables x across a panel of N cross-sectional units over T time periods (Pesaran & Shin, 1999; Pesaran et al., 2001). Before applying such models to the panel data, one has to first determine if a long-run equilibrium relationship exists between the variables using a cointegration test. However, panel data often suffers from the problem of cross-sectional dependence (CD/CSD), i.e. the units of observations are not independent of one another, and this is typically caused by unobserved common factors that affect all units (e.g. global financial crisis in the case of investment flows) (Pesaran, 2021). Traditional residuals-based tests like the Pedroni panel cointegration test enforce strict exogeneity constraints and are not robust to cross-sectional dependence, so they are vulnerable to over-rejection (Pedroni, 2004). In cases of cross-sectional dependence, tests based on structural dynamics like the Westerlund (2007) test have to be used, which is now only available on Stata (Persyn & Westerlund, 2008).

This paper introduces a Python and R functional approximation implementation of the Westerlund (2007) tests, which can be accessed at https://github.com/bosco-hung/WesterlundTest. The Python version is also available at PyPI (https://pypi.org/project/Westerlund/) and the R version is currently under review. The implementations provide similar functionalities of the Stata command `xtwest` developed by Persyn and Westerlund (2008), including the calculation of the four primary test statistics (G_τ, G_α, P_τ, P_α) through asymptotic standardization using moments from Westerlund (2007), and a bootstrap procedure to handle cross-sectional dependence. An additional bootstrapping visualization function is added to facilitate the examination of the bootstrap distribution and significance of the statistics of interest.

This paper starts by discussing the econometric model developed by Westerlund (2007). Then, it discusses the Python and R implementations. It proceeds with some points to note and closes with some implementation examples.

2 The Econometric Model

The Westerlund test is based on a structural equation known as the Conditional Error Correction Model (ECM):

$$\Delta y_{it} = \underbrace{\delta_i' d_t}_{\text{Deterministics}} + \underbrace{\alpha_i (y_{i,t-1} - \beta_i' x_{i,t-1})}_{\text{Long-Run Equilibrium}} + \underbrace{\sum_{j=1}^{p_i} \alpha_{ij} \Delta y_{i,t-j} + \sum_{j=-q_i}^{p_i} \gamma_{ij} \Delta x_{i,t-j}}_{\text{Short-Run Dynamics}} + e_{it} \tag{1}$$

where its core lies in the long-run equilibrium term $\alpha_i(y_{i,t-1} - \beta_i' x_{i,t-1})$. The expression $y_{i,t-1} - \beta_i' x_{i,t-1}$ represents the deviation from equilibrium in the previous time period. If variables y and x are cointegrated, they move together in the long run according to the relationship $y_{it} = \beta_i x_{it}$. Therefore, the residual of $y_{i,t-1} - \beta_i' x_{i,t-1}$ represents the "error" or "disequilibrium" at time $t-1$. The parameter α_i determines how the system responds to that disequilibrium. If $\alpha_i < 0$ (Error Correction), the system is stable. If y_{t-1} was "too high" relative to x_{t-1} (positive error), the negative α_i forces Δy_{it} to be negative. The variable y falls to restore equilibrium. On the other hand, if $\alpha_i = 0$ (No Error Correction), the change in y (Δy_{it}) does not depend on the previous level. The variables drift apart indiscriminately. This implies no cointegration.

The summation terms $\sum_{j=1}^{p_i} \phi_{ij} \Delta y_{i,t-j} + \sum_{j=-q_i}^{p_i} \gamma_{ij} \Delta x_{i,t-j}$ represent the short-run shocks. These terms $\sum_{j=1}^{p_i} \phi_{ij} \Delta y_{i,t-j}$ capture the inertia in the system. By including sufficient lags of Δy and Δx, we ensure that the regression residual e_{it} is free of serial correlation (white noise), so as to ensure the validity of the OLS t-statistics. On the other hand, the term $\sum_{j=-q_i}^{0} \gamma_{ij} \Delta x_{i,t-j}$ includes current and future differences of x for correcting for endogeneity. By projecting the error onto the leads and lags of Δx, the regressors become strictly exogenous with respect to the error term, allowing for valid asymptotic inference on α_i.

The term $\delta_i' d_t$ accounts for trends in the data that are not related to the stochastic relationship between x and y. In Case 1 $(d_t = 0)$, there is no intercept. The data has a zero mean; In Case 2 $(d_t = \{1\})$, there is a constant intercept. The data fluctuates around a non-zero level; Finally, in Case 3 $(d_t = \{1, t\})$, there is both an intercept and linear trend. The data drifts upward or downward over time.

2.1 Hypothesis Testing Framework

The statistical test tests the value of α_i derived from the equation outlined in the following. Under the Null Hypothesis (H_0), there is no error correction, i.e. there is no cointegration exists for any unit:

$$H_0 : \alpha_i = 0 \quad \text{for all } i = 1 \ldots N \tag{2}$$

Under the null, there is no error correction, i.e. there is no cointegration exists for any unit.

The Alternative Hypothesis (H_1) depends on which statistic is used (Group vs. Panel): In the case of group mean statistics (G_τ, G_α), they do not assume a common speed of adjustment. They test whether cointegration exists for at least one unit in the panel:

$$H_1^G : \alpha_i < 0 \quad \text{for at least one } i \tag{3}$$

In the case of panel mean statistics (P_τ, P_α), they "pool" the information across the cross-sectional dimension and test whether cointegration exists for the entire panel as a whole, assuming a homogeneous speed of adjustment:

$$H_1^P : \alpha_i = \alpha < 0 \quad \text{for all } i \tag{4}$$

3 R

The R version of the code features a main driver: `westerlund_test`. This primary interface connects the functions to perform input validation and data sorting, as well as to handle the branching logic between asymptotic and bootstrap inference.

The implementation enforces strict time-series continuity by checking for time gaps:

$$\Delta t = t_{i,s} - t_{i,s-1} \tag{5}$$

If $\Delta t > 1$, a "hole" is identified, and the function halts to prevent invalid differencing. The data is sorted to ensure the vector Y follows a block-unit structure:

$$Y = [y_{1,1}, y_{1,2}, \dots, y_{1,T}, \quad y_{2,1}, \dots, y_{2,T}, \quad \dots, \quad y_{N,T}]' \tag{6}$$

3.1 Econometric Estimation Logic

The `WesterlundPlain` function estimates ECMs and aggregates them into panel statistics.

3.1.1 Unit-Level Estimation

For each unit i, the code estimates the unrestricted ECM outlined in Equation 1. If `auto` selection is enabled, the code minimizes the Akaike Information Criterion (AIC). If `westerlund=TRUE`, it uses the specific penalty:

$$AIC(p,q) = \ln\left(\frac{RSS_i}{T_i - p - q - 1}\right) + \frac{2(p + q + \det + 1)}{T_i - p_{max} - q_{max}} \tag{7}$$

3.1.2 Long-Run Variance (HAC) Estimation

The helper `calc_lrvar_bartlett` computes the Newey-West variance using a Bartlett kernel. Following the Stata convention, autocovariances $\hat{\gamma}_j$ are scaled by $1/n$:

$$\hat{\omega}^2 = \hat{\gamma}_0 + 2\sum_{j=1}^{M}\left(1 - \frac{j}{M+1}\right)\hat{\gamma}_j, \quad \hat{\gamma}_j = \frac{1}{n}\sum_{t=j+1}^{n} \hat{e}_t \hat{e}_{t-j} \tag{8}$$

3.1.3 Panel Statistics Construction

3.1.3.1 Group Mean Statistics (G) The group mean statistics (G) are computed by averaging individual unit results. Following the completion of the unit-level ECMs, the code aggregates the individual speed-of-adjustment coefficients ($\hat{\alpha}_i$) and their standard errors:

$$G_\tau = \frac{1}{N}\sum_{i=1}^{N}\frac{\hat{\alpha}_i}{SE(\hat{\alpha}_i)} \quad \text{and} \quad G_\alpha = \frac{1}{N}\sum_{i=1}^{N}\frac{T_i\hat{\alpha}_i}{\hat{\alpha}_i(1)} \tag{9}$$

where $\hat{\alpha}_i(1) = \frac{\hat{\omega}_{ui}}{\hat{\omega}_{yi}}$ is the ratio of long-run standard deviations derived from the Bartlett-kernel HAC estimation described earlier.

3.1.3.2 Pooled Panel Statistics (P) For pooled statistics (P_τ, P_α), the code partials out short-run dynamics and deterministics from Δy_{it} and $y_{i,t-1}$ using the average lag and lead orders calculated across all units. The pooled coefficient $\hat{\alpha}_{pooled}$ is then estimated by aggregating these filtered residuals:

$$\hat{\alpha}_{pooled} = \left(\sum_{i=1}^{N}\sum_{t=1}^{T}\tilde{y}_{i,t-1}^2\right)^{-1}\sum_{i=1}^{N}\sum_{t=1}^{T}\frac{1}{\hat{\alpha}_i(1)}\tilde{y}_{i,t-1}\Delta\tilde{y}_{it} \tag{10}$$

The final statistics are then constructed as:

$$P_\tau = \frac{\hat{\alpha}_{pooled}}{SE(\hat{\alpha}_{pooled})} \quad \text{and} \quad P_\alpha = T \cdot \hat{\alpha}_{pooled} \tag{11}$$

where T represents the effective sample size after accounting for the loss of observations due to the chosen lag and lead lengths11111111.

3.2 Standardization and Display

The function `DisplayWesterlund` converts raw statistics S into Z-scores using asymptotic moments μ_S and σ_S:

$$Z_S = \frac{\sqrt{N}(S - \mu_S)}{\sigma_S} \tag{12}$$

The moments are retrieved from hard-coded matrices indexed by deterministic cases (1: None, 2: Constant, 3: Trend) and the number of regressors K in the original Westerlund (2007) paper.

As the test is lower-tailed, the null of no cointegration is rejected if the observed statistic is significantly negative.

3.3 Bootstrap Inference: `WesterlundBootstrap`

As explained earlier, the Westerlund test allows the handling of cross-sectional dependence. To do so, the code implements a recursive sieve bootstrap under the null hypothesis ($H_0 : \alpha_i = 0$) following a multi-stage process:

1. **Null-Model Estimation:** For each unit i, the model is estimated under H_0. Residuals ($\hat{e}_{it}$) and regressor differences (Δx_{it}) are extracted and centered within each unit to ensure the bootstrap innovations have a zero mean.

2. **Synchronized Temporal Shuffle:** To preserve the contemporaneous correlation structure $E[e_{it}e_{jt}] \neq 0$, the temporal realignment must be identical across all cross-sectional units. A cluster bootstrap is performed on the time dimension to draw a vector of T time-block indices. Then, a single vector of random draws $U \sim (0, 1)$ is generated as a global shuffle key. Stable sort (using the original index as a tie-breaker) is then applied to this key. Every cross-sectional unit is re-indexed according to this identical permutation, so as to ensure that the cross-sectional "link" between units i and j at time t remains intact in the simulated data.

3. **Innovation Construction (u^*_{it}):** The bootstrap innovation is constructed by combining the reshuffled residuals e^*_{it} with the lags and leads of the centered regressor differences:

$$u^*_{it} = e^*_{it} + \sum_{j=-q}^{p} \hat{\gamma}_{ij} \Delta x^*_{i,t-j} \tag{13}$$

   With the `shiftNA` helper function, the calculation follows an "All-or-Nothing" missing value logic. If any component (residual or lagged difference) is `NA` due to time-gap sensitivity or boundary shifts, the entire observation u^*_{it} is treated as `NA`. Only after the summation is complete are these missing values converted to zeros to initialize the recursion. This prevents artificial inflation of the sample size and ensures the bootstrap's variance-covariance structure is not diluted by "zero-filling" missing data too early.

4. **Recursive Simulation:** The series of first-differences Δy^*_{it} is generated recursively using the unit-specific autoregressive coefficients $\hat{\phi}_{ij}$:

$$\Delta y^*_{it} = \sum_{j=1}^{p} \hat{\phi}_{ij} \Delta y^*_{i,t-j} + u^*_{it} \tag{14}$$

   Initialization values for the first p periods are handled by treating them as missing values (`NA`) to replicate the finite-sample behavior of the original test.

5. **Integration:** The simulated differences are integrated to recover the level variables for both y and x using the `cumsum` function:

$$y^*_{it} = \sum_{s=1}^{t} \Delta y^*_{is}, \quad X^*_{it} = \sum_{s=1}^{t} \Delta X^*_{is} \tag{15}$$

   Rows containing `NA` values (due to lags or padding) are dropped before the final panel is re-estimated using the `WesterlundPlain` routine.

Finally, the Robust p-value is computed as:

$$p^* = \frac{\sum_{b=1}^{B} I(Stat^*_b \leq Stat_{obs}) + 1}{B + 1} \tag{16}$$

where finite sample corrections are applied.

3.4 Visualization

The R implementation provides a `plot_westerlund_bootstrap` helper to visualize the bootstrapping distribution. The function renders a 2×2 grid using `par(mfrow = c(2, 2))` for a side-by-side comparison of the group mean (G_τ, G_α) and panel (P_τ, P_α) statistics, allowing the researcher to observe if cointegration is supported by individual units or the panel as a whole. Mathematically, if B is the number of bootstrap replications, the visualization plots the density function:

$$\hat{f}(s) = \frac{1}{Bh} \sum_{b=1}^{B} K\left(\frac{s - Stat^*_b}{h}\right) \tag{17}$$

where $Stat^*_b$ are the simulated statistics, $K(\cdot)$ is the kernel function, and h is the bandwidth.

The empirical null distribution is generated using the `density()` function with a Gaussian kernel. The area under the curve is typically shaded to represent the probability mass of the null hypothesis. A solid line represents the observed statistic ($Stat_{obs}$). A dashed line represents the 5% left-tail critical value (CV^*). If the Kernel Density Estimation (KDE) mass is located far to the right of the observed statistic, the result is robust; if they overlap significantly, the asymptotic significance may be a false positive caused by cross-sectional dependence.

4 Python

A Python version of the Westerlund Test framework is also developed to accommodate the preferences of different users. The WesterlundTest class provides a modular framework for evaluating cointegration in panel data.

Similarly to the R version, the Python method has a built-in gap-aware logic that does not rely on simple position-based shifting. The methods _get_ts_map and _get_lag_lead form the core of the gap-aware logic. The class maps values to their specific time index T. When a lag k is requested, the engine looks for the value at $T-k$. If the time index is discontinuous, the engine assigns a NA to that observation to prevent the model from erroneously calculating short-run dynamics across missing time periods.

4.1 Econometric Estimation Logic

4.1.1 Information Criterion and Model Selection

The _selection_loop identifies the optimal lag (p) and lead (q) lengths. For the specialized Westerlund criterion, the penalty function is defined the same way in R explained in Equation 7.

4.1.2 Long-Run Variance Estimation

The method _lrvar_kernel implements a Bartlett kernel (HAC) estimator which scales the same way as in the R version explained in Equation 8.

4.1.3 The Error Correction Model (ECM)

The _get_optimal_model method then fits the unrestricted ECM regression outlined in Equation 1.

4.1.4 Panel Statistics Construction

The _run_westerlund_plain method executes two estimation passes and estimates the group mean statistics and panel statistics the same way as outlined in Equations 9 and 11.

4.2 Bootstrap Inference: _bootstrap_run

The _bootstrap_run method accounts for cross-sectional dependence through a recursive sieve bootstrap conducted under the null hypothesis $(H_0 : \alpha_i = 0)$. The procedure follows the following steps:

1. **Null-Model Estimation:** For each cross-sectional unit i, the ECM is estimated with the null of no cointegration imposed. Residuals ($\hat{e}_{it}$) and regressor differences (ΔX_{it}) are extracted and centered. The short-run autoregressive coefficients ($\hat{\phi}_{ij}$) and dynamic regressor coefficients ($\hat{\gamma}_{ij}$) are stored to preserve the unique dynamics of each unit.

2. **Synchornized Stable Shuffle:** To maintain contemporaneous correlation $E[e_{it}e_{jt}] \neq 0$, the temporal realignment must be identical across the panel. A single vector of random draws is generated for the entire iteration. This vector ensures that the shuffle is not averaged toward a mean but remains a true shared permutation. A stable-sort permutation (`kind="mergesort"`) is then applied to this key to prevent arbitrary reordering of ties, ensuring that every unit in the panel is subjected to the exact same temporal realignment.

3. **Innovation Construction** (u^*_{it})**:** Composite bootstrap innovations are generated by combining the resampled residuals e^*_{it} with the estimated dynamics of the resampled regressor differences, as outlined in Equation 13. To match the finite-sample variance of the original test, the code implements the same NA logic outlined earlier: If a lagged or led regressor difference is `NaN` (due to boundary shifts or time-gaps), the entire observation u^*_{it} becomes `NaN`. These values are only converted to zero (0.0) immediately before the recursive step to ensure the AR recursion has a clean initialization.

4. **Recursive Simulation and Integration:** The bootstrap series are reconstructed in a two-stage process. First, Δy is solved recursively. Second, both the dependent variable and regressors are integrated via cumulative summation (`np.cumsum`) to recover levels the same way outlined earlier in Equations 14 and 15. Initialization values are handled by setting the first p observations to `NaN` to match the finite-sample behavior of the original test.

5. **Re-Estimation:** The full Westerlund test is executed on the newly simulated panel $\{y^*, X^*\}$. This process is repeated for B replications to generate the empirical distribution of the G_t, G_a, P_t, and P_a statistics under the null.

4.3 Standardization and Display

Standardized Z-scores are calculated in `_display_final` using the same way as explained in Equation 12.

4.4 Visualization

Similarly to R, the Python implementation also provides a visualization tool. It employs the `gaussian_kde` from `scipy.stats` to estimate the density, and then uses `ax.fill_between` to color the area under the KDE curve. This visually represents where the statistics would fall if no cointegration existed.

5 Points to Note

There remains some implementation-level differences that would lead to minor discrepancies between the results given by the original `xtwest` function on Stata designed by Persyn and Westerlund (2008), and the Python and R versions outlined in this paper. If you run the asymptotic test (no bootstrap), the results should be nearly identical (up to floating-point precision). If you run the bootstrap, the robust p-values will vary slightly. One should be careful about the marginally significant cases if the bootstrapping times B is low. I provide some discussion of the known primary sources of differences and the implemented mitigation below.

5.1 Randomization

The Random Number Generation (RNG) mechanisms in these languages, including Stata, are fundamentally different and difficult to align. RNG is used when cross-sectional dependence is handled during the bootstrap "shuffling" phase. The R implementation uses the standard `Mersenne-Twister` algorithm via `RNGkind`. On the other hand, the Python implementation uses the `numpy.random.Generator` with the `MT19937` core. Even with the same seed, the bit-stream of a Mersenne-Twister generator differs between R and Python's NumPy, meaning the resampled residuals (u_{it}^*) will not be identical and the resulting robust p-values will be slightly different.

5.2 OLS Equations

The two languages use different underlying linear algebra libraries for solving the OLS equations in the unit-specific ECMs. R (stats::lm) uses QR decomposition, as it is highly stable and handles near-collinear regressors by pivoting. On the other hand, Python (statsmodels.OLS) by default uses the Moore-Penrose pseudoinverse (`pinv`) to solve the least squares problem. I manually set it to use QR to align the results with R better.

5.3 Floating Point Drifts

Since the levels are built via integration and recursion, tiny differences are magnified.

6 Implementation Examples

In the following, I provide the results of the Python and R implementation replicating the results recorded by Persyn and Westerlund (2008). In the asymptotic cases, the results are almost perfectly similar, subject to display differences. In the robust cases, there are some minor discrepancies. Tables 1, 2, and 3 provide the results given by Python, and Figure 1 gives the corresponding bootstrapping visualization. Tables 4, 5, and 6 provide the results given by R, and Figure 2 gives the corresponding bootstrapping visualization. Seed 123 is used in both cases.

Table 1: *Python Replication of Table 7 in Westerlund (2007)*

With 20 series and 1 covariate
Average AIC selected lag length: 2.80
Average AIC selected lead length: 1.65

Statistic	Value	Z-value	p-value
Gt	-4.082	-9.613	0.000
Ga	-27.702	-10.626	0.000
Pt	-12.969	-4.100	0.000
Pa	-22.470	-10.119	0.000

Table 2: *Python Replication of the First Table in p.240 in Persyn and Westerlund (2008)*

Westerlund ECM Panel Cointegration Tests
Series: loghex ~ loggdp
N (Groups): 20
Lags: 1-1 | Leads: 1-1 | Window: 3

Statistic	Value	Z-score	p-value	Robust P
Gt	-2.681	-1.734	0.041	0.065
Ga	-10.927	0.714	0.762	0.111
Pt	-12.035	-2.961	0.002	0.040
Pa	-10.524	-1.160	0.123	0.059

Table 3: *Python Replication of the second table in p.240 in Persyn and Westerlund (2008)*

Westerlund ECM Panel Cointegration Tests
Series: loghex ~ loggdp
N (Groups): 20
Lags: 1-2 | Leads: 0-1 | Window: 2

Statistic	Value	Z-score	p-value	Robust P
Gt	-2.736	-2.036	0.021	–
Ga	-11.253	0.499	0.691	–
Pt	-12.859	-3.903	0.000	–
Pa	-11.773	-2.072	0.019	–

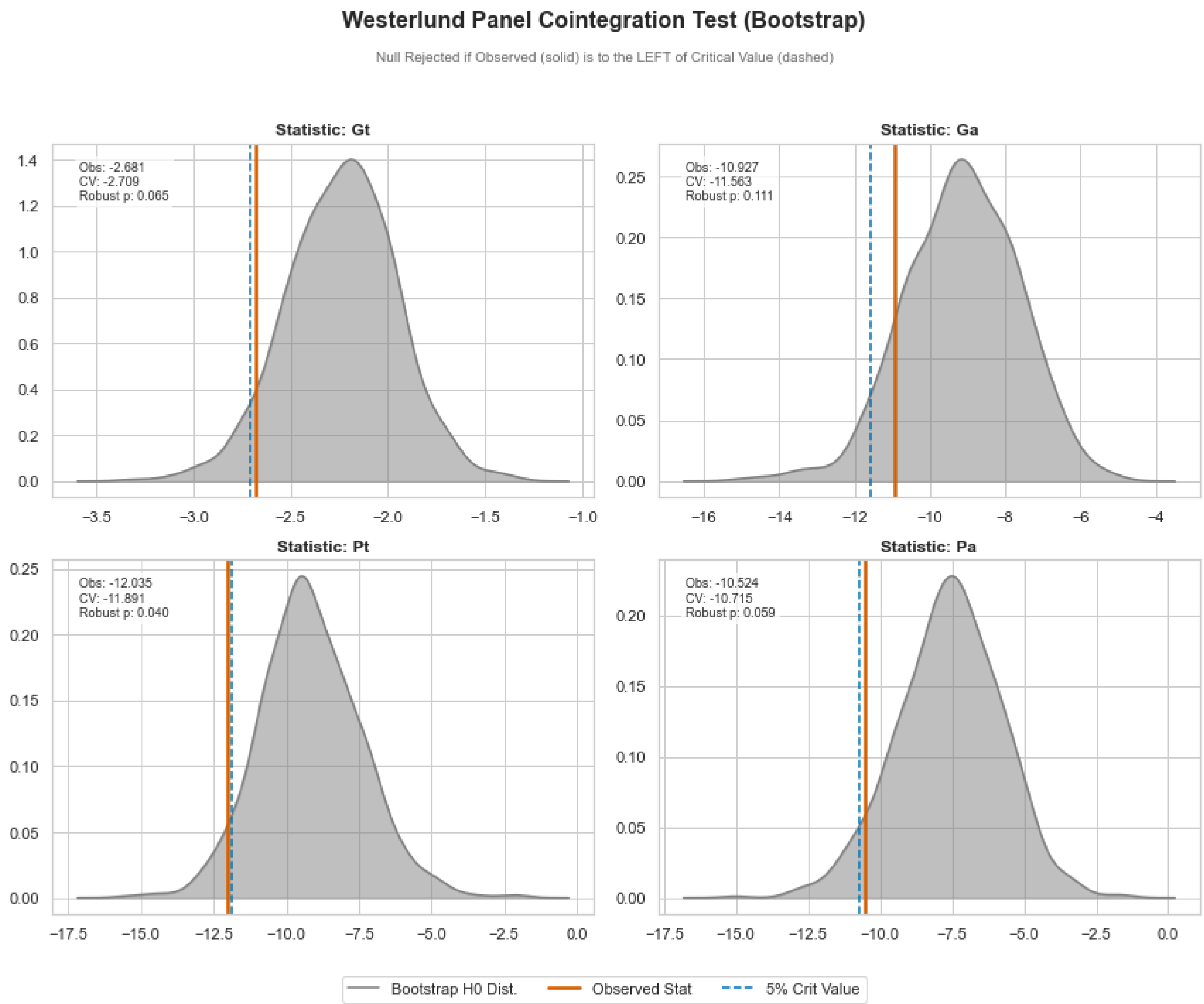

Figure 1: *Bootstrapping Distribution: Python Replication of the First Table in p.240 in Persyn and Westerlund (2008)*

Table 4: *R Replication of Table 7 in Westerlund (2007)*

With 20 series and 1 covariate

Average AIC selected lag length: 2.80

Average AIC selected lead length: 1.65

Statistic	Value	Z-value	p-value
Gt	−4.082	−9.613	0.000
Ga	−27.702	−10.626	0.000
Pt	−12.969	−4.100	0.000
Pa	−22.470	−10.119	0.000

Table 5: *R Replication of the First Table in p.240 in Persyn and Westerlund (2008)*

Westerlund ECM Panel Cointegration Tests

Series: loghex ~ loggdp

N (Groups): 20

Lags: 1-1 | Leads: 1-1 | Window: 3

Statistic	Value	Z-score	p-value	Robust P
Gt	−2.681	−1.731	0.042	0.055
Ga	−10.927	0.713	0.762	0.100
Pt	−12.035	−2.959	0.002	0.029
Pa	−10.524	−1.160	0.123	0.054

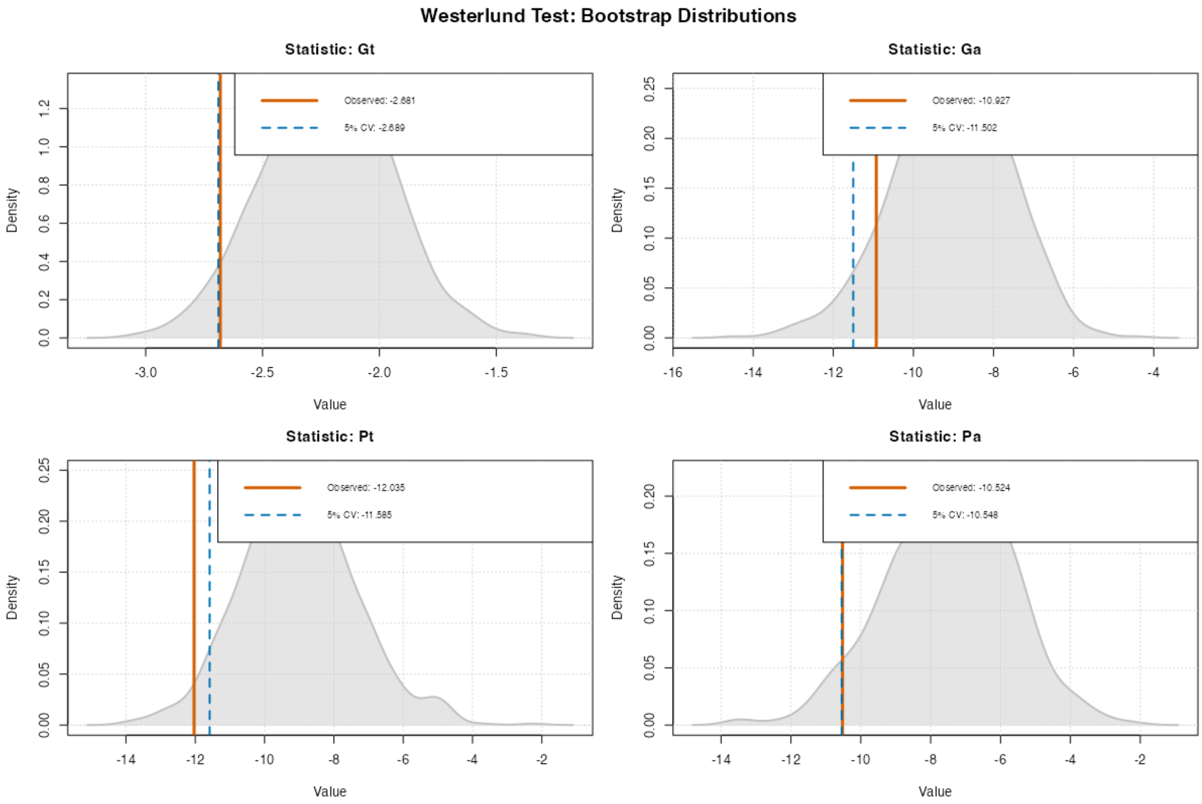

Figure 2: *Bootstrapping Distribution: R Replication of the First Table in p.240 in Persyn and Westerlund (2008)*

Table 6: *R Replication of the second table in p.240 in (Persyn & Westerlund, 2008)*

Westerlund ECM Panel Cointegration Tests
Series: loghex ~ loggdp
N (Groups): 20
Lags: 1-2 | Leads: 0-1 | Window: 2

Statistic	Value	Z-score	*p*-value	Robust P
Gt	−2.736	−2.033	0.021	–
Ga	−11.253	0.499	0.691	–
Pt	−12.859	−3.902	0.000	–
Pa	−11.773	−2.072	0.019	–

7 Bibliography

References

Pedroni, P. (2004). Panel cointegration: Asymptotic and finite sample properties of pooled time series tests with an application to the ppp hypothesis. *Econometric Theory*, *20*(03). https://doi.org/10.1017/S0266466604203073

Persyn, D., & Westerlund, J. (2008). Error-Correction–Based Cointegration Tests for Panel Data. *The Stata Journal: Promoting communications on statistics and Stata*, *8*(2), 232–241. https://doi.org/10.1177/1536867X0800800205

Pesaran, M. H. (2021). General diagnostic tests for cross-sectional dependence in panels. *Empirical Economics*, *60*(1), 13–50. https://doi.org/10.1007/s00181-020-01875-7

Pesaran, M. H., & Shin, Y. (1999). An Autoregressive Distributed-Lag Modelling Approach to Cointegration Analysis. In S. Strom (Ed.), *Econometrics and Economic Theory in the 20th Century* (pp. 371–413). Cambridge University Press. https://doi.org/10.1017/CCOL521633230.011

Pesaran, M. H., Shin, Y., & Smith, R. J. (2001). Bounds testing approaches to the analysis of level relationships. *Journal of Applied Econometrics*, *16*(3), 289–326. https://doi.org/10.1002/jae.616

Westerlund, J. (2007). Testing for Error Correction in Panel Data*. *Oxford Bulletin of Economics and Statistics*, *69*(6), 709–748. https://doi.org/10.1111/j.1468-0084.2007.00477.x